



# PRD – Lógica – Tema 6 – Sumar cifras de un número



## Resumen ejecutivo

Este documento define los requisitos para el desarrollo del **Tema 6 – Sumar cifras de un número**, correspondiente al Segundo Parcial / Trabajo Práctico de Lógica (Fundamentos de pseudocódigo y programación básica en Python).

El programa debe calcular la suma de las cifras de un número entero ingresado por el usuario, utilizando exclusivamente operaciones aritméticas (división entera y resto), sin convertir el número a texto ni utilizar estructuras de datos como listas o librerías externas.



## Descripción del problema

Dado un número entero `n` mayor o igual que cero, el programa debe calcular la suma de todas sus cifras individuales.

**Ejemplo:** Si `n = 384`, entonces `3 + 8 + 4 = 15`.

La resolución debe hacerse extrayendo cada dígito mediante operaciones de **división entera** (`//`) y **resto** (`%`), iterando con un `while` hasta que el número se agote.

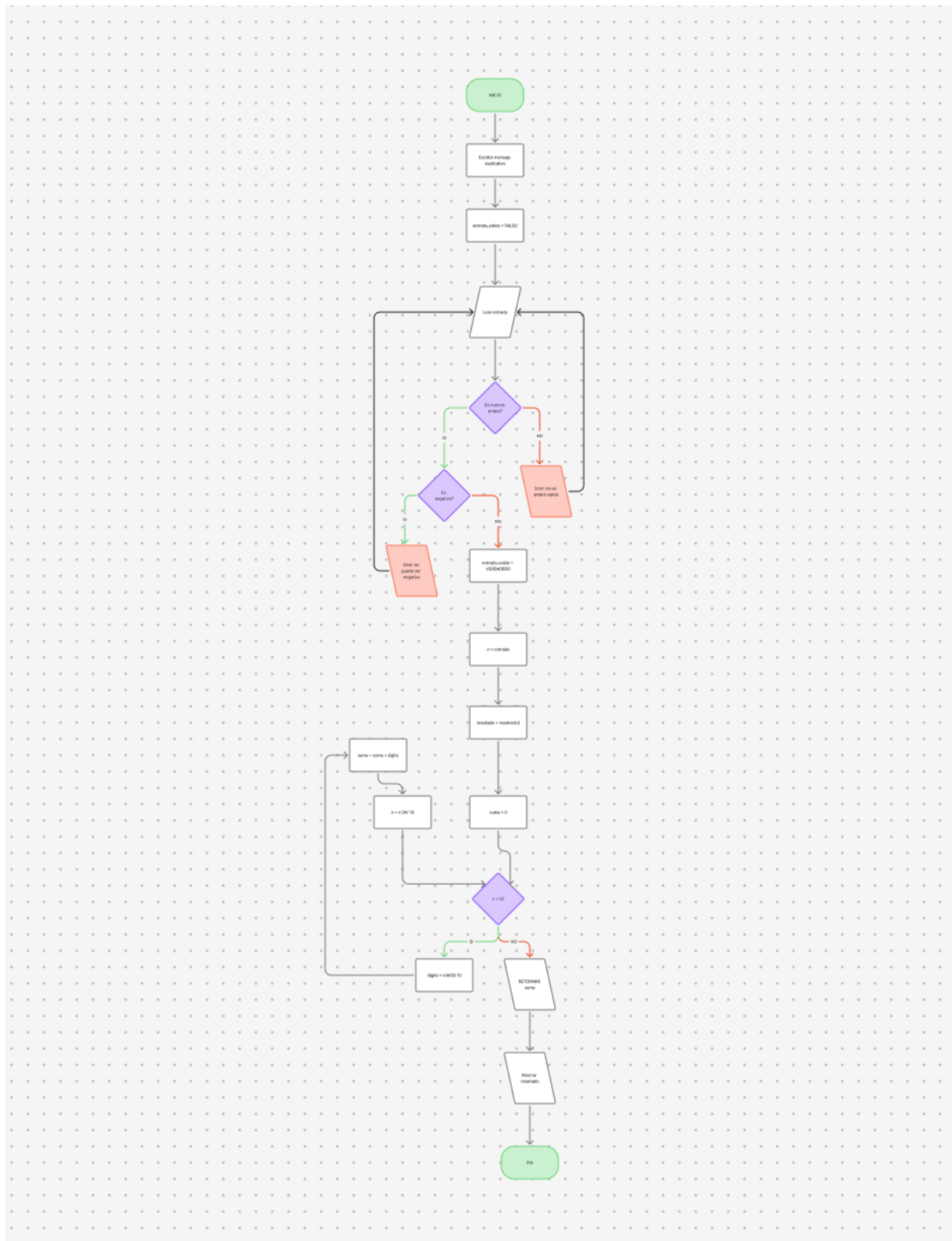


## Diagrama de Flujo

El siguiente diagrama representa visualmente el flujo completo del algoritmo, incluyendo la validación de entrada del programa principal y la función

`resolver(n)`.

<https://www.figma.com/board/QXjjWmdUvxTb6WCMeljbT4/Diagrama-de-Flujo---Sumar-cifras-de-un-numero?node-id=0-1&t=de21R0h6ecHgD2Dz-1>



## Pseudocódigo

A continuación se describe el algoritmo en pseudocódigo, previo a la implementación en Python.

## Función `resolver(n)`

```
FUNCIÓN resolver(n):  
    suma ← 0  
  
    MIENTRAS n > 0 HACER:  
        digito ← n MOD 10      // Extraer el dígito menos  
significativo  
        suma ← suma + digito  // Acumular el dígito  
        n ← n DIV 10          // Eliminar el dígito ya pr  
oceso  
    FIN MIENTRAS  
  
    RETORNAR suma  
FIN FUNCIÓN
```

## Programa principal (ejecución por consola)

```
INICIO  
  
    ESCRIBIR "Este programa calcula la suma de las cifras d  
e un número entero."  
  
    entrada_válida ← FALSO  
  
    MIENTRAS entrada_válida = FALSO HACER:  
        ESCRIBIR "Ingresá un número entero mayor o igual a  
cero: "  
        LEER entrada  
  
        SI entrada NO es un número entero ENTONCES:  
            ESCRIBIR "Error: el valor ingresado no es un nú  
mero entero válido."  
        SINO SI entrada < 0 ENTONCES:  
            ESCRIBIR "Error: el número no puede ser negativ  
o."  
        SINO:  
            entrada_válida ← VERDADERO
```

FIN SI  
FIN MIENTRAS

$n \leftarrow \text{entrada}$   
 $\text{resultado} \leftarrow \text{resolver}(n)$

ESCRIBIR "La suma de las cifras de " +  $n$  + " es: " + resultado

FIN

## Seguimiento del algoritmo — Ejemplo con $n = 384$

| Iteración | $n$ (antes) | $\text{digito} = n \text{ MOD } 10$ | suma<br>(acumulada) | $n = n \text{ DIV } 10$ |
|-----------|-------------|-------------------------------------|---------------------|-------------------------|
| 1         | 384         | 4                                   | 4                   | 38                      |
| 2         | 38          | 8                                   | 12                  | 3                       |
| 3         | 3           | 3                                   | 15                  | 0                       |
| —         | 0           | (fin del bucle)                     | 15                  | —                       |

Resultado esperado:  $\text{resolver}(384) \rightarrow 15$  ✓



## Especificación técnica

### Firma de la función

```
def resolver(n):
```

- **Parámetro de entrada:**  $n$  — número entero  $\geq 0$
- **Valor de retorno:** entero — la suma de las cifras de  $n$

### Estructuras de programación esperadas

| Estructura         | Uso                                         |
|--------------------|---------------------------------------------|
| <code>while</code> | Iterar mientras queden dígitos por procesar |
| Acumulador         | Sumar cada dígito extraído                  |

| Estructura                                 | Uso                                   |
|--------------------------------------------|---------------------------------------|
| Operador <code>%</code> (resto)            | Extraer el dígito menos significativo |
| Operador <code>//</code> (división entera) | Eliminar el dígito ya procesado       |

## ✓ Requisitos funcionales

### RF-01 — Función `resolver(n)`

- Debe recibir un entero `n`  $\geq 0$ .
- Debe devolver la suma de sus cifras como entero.
- Debe usar `while`, acumulador, `%` y `//` para resolver el cálculo.
- **No** puede usar `str(n)` ni ninguna conversión a texto.

### RF-02 — Ejecución desde consola

Cuando el programa se ejecute con `python tp_logica.py`, debe:

1. Explicar brevemente qué problema resuelve.
2. Pedir al usuario que ingrese un número entero.
3. Validar los datos ingresados.
4. Calcular la suma de cifras.
5. Mostrar el resultado con una frase clara y comprensible.

### RF-03 — Validación de datos

| Caso                                                | Comportamiento esperado                           |
|-----------------------------------------------------|---------------------------------------------------|
| El usuario ingresa letras o caracteres no numéricos | Mostrar mensaje de error y volver a pedir el dato |
| El usuario ingresa un número negativo               | Mostrar mensaje de error y volver a pedir el dato |
| El usuario ingresa <code>0</code>                   | Aceptarlo (es un caso válido: suma = 0)           |

## ⊘ Restricciones de implementación

### Permitido

- `if`, `else`, `elif`

- `for`
- `while`
- Contadores y acumuladores
- Operadores matemáticos, relacionales y lógicos
- Funciones propias simples
- `range()` dentro de un `for`

## No permitido para resolver el cálculo principal

- `str(n)` o cualquier conversión a texto
- Listas, diccionarios, conjuntos o tuplas como estructura de resolución
- Librerías externas
- `sum()`, `max()`, `min()` para el cálculo principal
- Comprensiones de listas
- Recursividad

## Lote mínimo de pruebas

| # | Entrada                | Resultado esperado | Comentario                                 |
|---|------------------------|--------------------|--------------------------------------------|
| 1 | <code>n = 0</code>     | <code>0</code>     | Caso límite: número cero                   |
| 2 | <code>n = 7</code>     | <code>7</code>     | Caso pequeño: número de un dígito          |
| 3 | <code>n = 384</code>   | <code>15</code>    | Caso del enunciado: <code>3 + 8 + 4</code> |
| 4 | <code>n = 9999</code>  | <code>36</code>    | Caso mediano: todos nueves                 |
| 5 | <code>n = -5</code>    | Error              | No se aceptan negativos                    |
| 6 | <code>n = "abc"</code> | Error              | No se aceptan letras                       |

## Criterios de evaluación

| Criterio                            | Puntaje | Descripción                                                |
|-------------------------------------|---------|------------------------------------------------------------|
| Funcionamiento algorítmico          | 6 pts   | El programa resuelve correctamente el problema             |
| Informe                             | 2 pts   | Explica el problema, el algoritmo y las estructuras usadas |
| Experiencia de usuario y validación | 1 pt    | Guía al usuario y controla errores de entrada              |
| Lote de pruebas                     | 1 pt    | Presenta pruebas con resultados esperados y obtenidos      |

## Detalle — Funcionamiento algorítmico (6 pts)

| Aspecto                                                             | Puntaje |
|---------------------------------------------------------------------|---------|
| Implementa correctamente <code>resolver(n)</code>                   | 1.5 pts |
| Usa correctamente <code>while</code>                                | 1 pt    |
| Usa acumulador correctamente                                        | 1 pt    |
| Devuelve el tipo de dato esperado (entero)                          | 0.5 pts |
| Maneja casos límite correctamente ( <code>n = 0</code> , un dígito) | 1 pt    |



## Entregables requeridos

| Entregable                | Descripción                                                                    |
|---------------------------|--------------------------------------------------------------------------------|
| <code>tp_logica.py</code> | Archivo Python con la función <code>resolver(n)</code> y ejecución por consola |
| Informe (PDF o DOCX)      | Explicación del problema, estructuras usadas, código y pruebas                 |
| Lote de pruebas           | Mínimo 5 casos con entradas, resultados esperados y obtenidos                  |



## Contenido del informe

El informe debe incluir:

1. Nombre de los integrantes del grupo
2. Tema asignado
3. Explicación del problema con sus propias palabras

4. Explicación de las estructuras utilizadas: `while`, acumulador
  5. Explicación breve del código implementado
  6. Capturas o ejemplos de ejecución desde consola
  7. Lote de pruebas completo
  8. En caso de haber usado IA: herramienta utilizada, preguntas realizadas, partes aceptadas, partes corregidas y decisiones propias
- 

## Consideraciones especiales

- El caso `n = 0` debe ser aceptado y retornar `0`.
- El caso de un número de un solo dígito (ej. `n = 7`) debe retornar ese mismo dígito.
- El programa debe tolerar múltiples intentos inválidos sin cerrarse.
- El resultado debe mostrarse siempre dentro de una frase comprensible, no como un número aislado.